Trabalho Prático N.º 1

Engenharia Informática

Laboratório de Programação

04/04/2026

Por:

Rui Teixeira, 2025155352

Índice

1. INTRODUÇÃO **3**

2. PROTOCOLO **4**

3. PROCEDIMENTO **5**

4. EXECUÇÃO **6**

1. CLASSE *MAIN* 6
2. CLASSE FILE 8
3. MODELO DE DADOS 9
   1. Classe Question *9*
   2. Classe Difficulty *12*
   3. Classe Type *13*
   4. Classe Answer *15*
4. CLASSE *GENERATOR* 16
   1. Classe Generator Extensions *20*
5. CLASSE *FORMAT* 24

5. DEMONSTRAÇÃO **26**

6. MOODLE **27**

1. Introdução

O objetivo principal deste trabalho é “criar em Python uma ferramenta que possibilite a conversão de perguntas de correção automática, seja de escolha múltipla ou outro formato, de uma estrutura de texto separado por vírgulas (ou pontos e vírgulas) para o formato XML reconhecido pelo Moodle.” – Protocolo do Trabalho Prático N.º 1.

Para tal, pretende-se, em primeiro lugar, “desenvolver uma série de questões para correção automática, utilizando um formato de escolha múltipla ou outro, e organizá-las, por exemplo, num ficheiro de texto separado por vírgulas.” – Protocolo do Trabalho Prático N.º 1.

Python é uma linguagem de programação de alto-nível que se destaca pelo uso de “identação significativa”.

Caracteriza-se pela verificação de tipo e gestão de memória dinâmicos e suporta múltiplos paradigmas de programação tais como, Orientada a Objetos e Funcional.

2. Protocolo

Definiu-se ficheiro aceite o formato de dados de texto simples *Comma-separated Values* (*CSV*).

Desta forma as questões são armazenadas em tabela.

Cada campo é separado por ponto e vírgula e cada fila corresponde a um registo (Questão).

É permitida a submissão de um documento que contém as seguintes colunas:

1. *ID* – Identificador da questão;
2. *Difficulty* – Dificuldade da questão;
3. *Question* – Questão. É possível assinalar parágrafo através do símbolo " *...* ".

Exemplo: "Qual o resultado do seguinte conjunto de instruções? ... *a = [1, 2, 3]* ... *a.append(7)* ... *print(a)*";

4. *Question Type* – Tipo de questão ("*Multiple Choice*" ou "*True or False*");
5. *Wrong Answers* – Repostas erradas. Separadas pelo carácter "|";
6. *Correct Answers* - Repostas corretas. Separadas pelo carácter "|".

3. Procedimento

Procedeu-se à elaboração de 30 (trinta) questões sobre a linguagem de programação *Python*, distribuídas por 3 (três) níveis de dificuldade ("*Easy*", "*Intermediate*", "*Hard*"), com propostas de resultado "*Multiple Choice*" e "*True or False*".

As questões foram inseridas em tabela em formato *Excel*.

Foi exportado o conteúdo da referida tabela para ficheiro de texto (.*tx*t) separado por *tab*.

Substituiu-se o *tab* por ponto e vírgula e alterou-se a extensão do ficheiro, de texto (.*txt*) para *comma-separated values* (.*csv*).

Por último, inseriu-se o carácter "|" com o objetivo de separar as várias respostas.

4. Execução

1. Classe *Main*

1. Foram definidos. em primeiro lugar, os campos (*fields*) correspondentes ao tipo de ficheiro aceite para fonte e destino de dados:

```
questions_data_path_extension = ".CSV".upper()
generated_quiz_path_extension = ".XML".upper()
```

Figura 1: Tpo de ficheiro aceite para fonte e destino de dados.

2. Foram definidos campos correspondentes às localizações predefinidas da fonte e destino de dados:

```
questions_data_path = f"assets/QuestionsData{questions_data_path_extension}"
generated_quiz_path = f"quiz/Quiz{generated_quiz_path_extension}"
```

Figura 2: Localização de fonte e destino de dados.

3. Foi definida a classe *Main* e função (método) estática *run* responsável pela execução da aplicação:

```
class Main:  1 usage  Rui Teixeira *
    @staticmethod  1 usage  Rui Teixeira *
    def run() -> None:
```

Figura 3: Classe "Main".

4. É inicializada a variável *questions_data*, resultado da leitura do ficheiro de dados fornecido (caso válido).

Corresponde a uma lista de dicionários cuja chave e valor são *string*:

```
questions_data = File(
    Format.request_file_path(
        file_name: "Questions Data",
        questions_data_path_extension,
        questions_data_path)).read_file()
```

Figura 4: Inicialização de variável que detém dados fornecidos em ficheiro de tipo CSV.

5. É inicializada a variável *questions*, lista de objetos *Question*, correspondente à lista de questões fornecida:

```
questions = Question.create_questions(questions_data)
```

Figura 5: Inicialização de variável que detém lista de questões.

6. É instanciada a classe *Generator* e criado, na localização fornecida (caso válida), o ficheiro com as questões acima descritas, em formato definido pelo Moodle:

```
Generator(
    Format.request_file_path(
        file_name: "Generated Quiz",
        generated_quiz_path_extension,
        generated_quiz_path,
        must_exist: False)).generate_quiz(questions)
```

Figura 6: Criação de ficheiro com respetivas questões em formato de tipo XML.

2. Classe *File*

1. Foram definidos, em primeiro lugar, os campos correspondentes ao tipo de ficheiro aceite para leitura de dados pela classe. Tais como o delimitador de colunas e modo de leitura:

```
file_extension = ".CSV".upper()
file_delimiter = ';'
read_mode = "r"
```

Figura 7: Tipo de ficheiro aceite, delimitador e modo de leitura.

2. Foi definida a classe *File*, responsável pela leitura de ficheiro, tal como respetivo "construtor".

Recebe como parâmetro a localização do ficheiro:

```
class File:  4 usages  Rui Teixeira *
    def __init__(self: File, file_path: str):  Rui Teixeira
        self.file_path = file_path
```

Figura 8: Classe "File" e respetivo construtor.

3. Foi definida função *read_file*, a qual resulta numa lista de dicionários onde cada elemento corresponde a um registo da tabela.

É necessário que o ficheiro fornecido seja de tipo *CSV*:

```
def read_file(self: File) -> list[dict[str, str]]:  1 usage  Rui Teixeira *
    if not self.file_path.upper().endswith(file_extension):
        raise ValueError("Invalid File: '{}'. Required Comma-separated Values.".format(self.file_path))

    if not os.path.isfile(self.file_path):
        raise FileNotFoundError("File Not Found: '{}'.".format(self.file_path))

    with open(self.file_path, read_mode) as file:
        reader = csv.DictReader(file, delimiter=file_delimiter)
        return list(reader)
```

Figura 9: Funcão "read_file".

3. Modelo de Dados

1. Classe *Question*

1. Foram definidos, em primeiro lugar, os campos correspondentes a cada coluna necessária para criação de um objeto *Question*.

   Todos os campos devem constar em ficheiro fornecido:

```
id_tag = "ID"
difficulty_tag = "Difficulty"
text_tag = "Question"
type_tag = "Question Type"
wrong_answers_tag = "Wrong Answers"
correct_answers_tag = "Correct Answers"
```

Figura 10: Dados necessários à criação de objeto "Question".

2. Foi definida a classe *Question*:

```
@dataclass(frozen=True)  21 usages  Rui Teixeira
class Question:
    id: str
    difficulty: Difficulty
    text: str
    type: Type
    wrong_answers: list[Answer]
    correct_answers: list[Answer]
```

Figura 11: Classe "Question".

3. Foi definida função (método) estática *create_questions*.

   Recebe como parâmetro uma lista de dicionários cuja chave e valor são do tipo *string* e aos quais correspondem os dados de cada questão.

   Tem como resultado lista de objetos *Question*.

   Apenas são considerados objetos válidos:

```python
@staticmethod  1 usage  Rui Teixeira *
def create_questions(questions_data: list[dict[str, str]]) -> list[Question]:
    questions = []

    for question_data in questions_data:
        question = Question.__create_question_from_dictionary(question_data)

        if not Question.__is_question_valid(question):
            print(f"Required Fields: {id_tag}, {difficulty_tag}, {text_tag}, {type_tag}, {wrong_answers_tag}, {correct_answers_tag}.")
            print("Invalid Question: {}".format(question))
            continue

        questions.append(question)
    return questions
```

Figura 12: Função "create_questions".

4. Foi definida função (método) estática *create_question_from_dictionary*.

Recebe como parâmetro um dicionário ao qual corresponde uma questão.

Tem como resultado objeto *Question*:

```python
@staticmethod  1 usage  Rui Teixeira *
def __create_question_from_dictionary(question_data: dict[str, str]) -> Question:
    return Question(
        Question.__get_from_tag(question_data, id_tag),
        Difficulty.create_difficulty_from(Question.__get_from_tag(question_data, difficulty_tag)),
        Question.__get_from_tag(question_data, text_tag),
        Type.create_type_from(Question.__get_from_tag(question_data, type_tag)),
        Answer.create_answers(Question.__get_from_tag(question_data, wrong_answers_tag)),
        Answer.create_answers(Question.__get_from_tag(question_data, correct_answers_tag)))
```

Figura 13: Função "create_question_from_dictionary".

5. Foi definida função (método) estática *question_is_valid*.

Recebe como parâmetro uma questão a analizar.

Tem como resultado a veracidade da validade da questão.

Uma questão é considerada válida quando apresenta todos os campos necessários à construção do objeto e pelo menos uma resposta errada e uma resposta correta:

```
@staticmethod  1 usage  Rui Teixeira *
def __is_question_valid(question: Question) -> bool:
    return (question.id is not None
            and question.difficulty != DIFFICULTY_EMPTY
            and question.text is not None
            and question.type != TYPE_EMPTY
            and question.wrong_answers and len(question.wrong_answers) >= 1
            and question.correct_answers and len(question.correct_answers) >= 1)
```

Figura 14: Função "is_question_valid".

6. Foi definida função (método) estática *get_from_tag*.

Recebe como parâmetro um dicionário ao qual corresponde uma questão e qual a chave do valor a ser extraído.

Tem como resultado o valor correspondente à chave fornecida:

```
@staticmethod  6 usages  Rui Teixeira
def __get_from_tag(question_data: dict[str, str], tag: str) -> str:
    return question_data[tag] if tag in question_data else None
```

Figura 15: Função "ge_from_tag".

2. Classe *Difficulty*

1. Foi definida a classe *Difficulty*:

```
@dataclass(frozen=True)  11 usages  Rui Teixeira
class Difficulty:
    value: str
```

Figura 16: Classe "Difficulty".

2. Foi definida função (método) estática *create_difficulty_from*.

Recebe como parâmetro um valor para o qual se pretende criar um objeto *Difficulty*.

Tem como resultado objeto *Difficulty*.

Apenas são permitidos os valores definidos na lista *ALL*:

```
@staticmethod  1 usage  Rui Teixeira
def create_difficulty_from(value: str) -> Difficulty:
    for difficulty in ALL:
        if difficulty.value == value:
            return difficulty

    return EMPTY
```

Figura 17: Função "create_difficulty_from".

3. Foram inicializados campos auxiliares correspondentes aos valores permitidos:

```
EMPTY = Difficulty("")
EASY = Difficulty("Easy")
INTERMEDIATE = Difficulty("Intermediate")
HARD = Difficulty("Hard")
ALL = [EASY, INTERMEDIATE, HARD]
```

Figura 18: Campos Auxiliaries - "Difficulty".

3. Classe *Type*

1. Foi definida a class *Type*:

```
@dataclass(frozen=True)  13 usages  Rui Teixeira
class Type:
    value: str
```

Figura 19: Classe "Type".

2. Foi definida função (método) estática *create_type_from*.

Recebe como parâmetro um valor para o qual se pretende criar um objeto Type.

Tem como resultado objeto *Type*.

Apenas são permitidos os valores definidos na lista *ALL*:

```
@staticmethod  1 usage  Rui Teixeira
def create_type_from(value: str) -> Type:
    for type in ALL:
        if type.value == value:
            return type

    return EMPTY
```

Figura 20: Função "create_type_from".

3. Foi definida função (método) estática *translate_type*.

Recebe como parâmetro um objeto *Type*.

Tem como resultado objeto equivalente ao recebido, em formato aceite pelo Moodle:

```
@staticmethod  1 usage  Rui Teixeira
def translate_type(type: Type) -> Type:
    match type.value:
        case MULTIPLE_CHOICE.value:
            return TRANSLATED_MULTIPLE_CHOICE
        case TRUE_OR_FALSE.value:
            return TRANSLATED_TRUE_OR_FALSE
        case _:
            return type
```

Figura 21: Função "translate_type".

4. Foram inicializados campos auxiliaries correspondentes aos valores permitidos:

```
EMPTY = Type("")
MULTIPLE_CHOICE = Type("Multiple Choice")
TRUE_OR_FALSE = Type("True or False")
ALL = [MULTIPLE_CHOICE, TRUE_OR_FALSE]

TRANSLATED_MULTIPLE_CHOICE = Type("multichoice")
TRANSLATED_TRUE_OR_FALSE = Type("truefalse")
```

Figura 22: Campos Auxiliares - "Type".

4. Classe *Answer*

1. Foi definido, em primeiro lugar, o campo correspondente ao delimitador de respostas:

```
concatenated_answers_delimiter = "|"
```

Figura 23: Delimitador.

2. Foi definida a classe *Answer*:

```
@dataclass(frozen=True)  7 usages  Rui Teixeira
class Answer:
    value: str
```

Figura 24: Classe "Answer".

3. Foi definida função (método) estática *create_answers*.

Recebe como parâmetro respostas separadas pelo carácter “|”:

Tem como resultado lista de objetos *Answer*.

```
@staticmethod  2 usages  Rui Teixeira
def create_answers(concatenated_answers: str) -> list[Answer]:
    answers = concatenated_answers.split(concatenated_answers_delimiter)
    answers_list = []

    for answer in answers:
        answers_list.append(Answer(answer))

    return answers_list
```

Figura 25: Função "create_answers"

4. Classe *Generator*

1. Foram definidos, em primeiro lugar, os campos correspondentes ao tipo de ficheiro a contruir pela classe. Tais como modo de leitura e codificação:

```
file_extension = ".XML".upper()
write_bytes_mode = "wb"
default_encoding = "utf-8"
```

Figura 26: Tipo de ficheiro a construir, modo de leitura e codificação.

2. Foram definidos campos referentes a elementos a ser preenchidos na estrutura a incluir em ficheiro *XML*, em formato aceite pelo Moodle.

Todos os elementos são incluídos na raíz (*root*), identificada por *quiz*:

```
root_tag = "quiz"
difficulty_tag = "category"
question_tag = "question"

course_path_prefix = "$course$/"
```

Figura 27: Principais elementos constituíntes de ficheiro resultante.

3. Foi definida a class *Generator*, responsável pela construção e escrita de ficheiro aceite por “banco de questões” Moodle, tal como respetivo construtor.

Recebe como parâmetro a localização pretendida do ficheiro:

```
class Generator:  8 usages  Rui Teixeira *
    def __init__(self: Generator, quiz_path: str):  Rui Teixeira
        self.quiz_path = quiz_path
```

Figura 28: Classe "Generator" e respetivo construtor.

4. Foi definida função *generate_quiz*.

Recebe como parâmetro uma lista de objetos *Question* a incluir em ficheiro *XML*.

É inicializada a variável *root*, que corresponde à “árvore de elementos” posteriormente transferida para a função *write_file*, responsável pela escrita desta em ficheiro:

```
def generate_quiz(self: Generator, questions: list[Question]) -> None:  1 usage  Rui Teixeira
    root = self.__create_element_tree(questions)
    self.__write_file(root)
```

Figura 29: Função "generate_quiz".

5. Foi definida função (método) estática *create_element_tree* responsável pela construção de “árvore de elementos” a incluir em ficheiro *XML*.

As questões são inicialmente organizadas por dificuldade de modo a ser possível a definição de uma categoria, à qual corresponde a dificuldade, após o qual são construídos os elementos correspondentes a cada questão.

Recebe como parâmetro uma lista de objetos *Question*.

Tem como resultado “árvore de elementos”:

```
@staticmethod  1 usage  Rui Teixeira
def __create_element_tree(questions: list[Question]) -> ET.Element[str]:
    root = ET.Element(root_tag)

    questions_by_difficulty = Generator.__group_questions_by_difficulty(questions)

    for difficulty in DIFFICULTY_ALL:
        if difficulty in questions_by_difficulty:
            Generator.__create_difficulty_tag(root, difficulty)

            for question in questions_by_difficulty[difficulty]:
                Generator.__create_question_tag(root, question)

    Format.indent_element_tree(root)

    return root
```

Figura 30: Função "create_element_tree".

6. Foi definida função (método) estática *group_questions_by_difficulty*.

Recebe como parâmetro a lista de objetos *Question*.

Tem como resultado um dicionário onde a cada chave corresponde uma lista de questões, organizando as questões por dificuldade:

```
@staticmethod  1 usage  Rui Teixeira
def __group_questions_by_difficulty(questions: list[Question]) -> defaultdict[Difficulty, list[Question]]:
    questions_by_difficulty = defaultdict(list)
    for question in questions:
        questions_by_difficulty[question.difficulty].append(question)

    return questions_by_difficulty
```

Figura 31: Função "group_questions_by_difficulty".

7. Foi definida função (método) estática *create_difficulty_tag*.

Recebe como parâmetro a estrutura de elementos e a dificultade a incluir:

```
@staticmethod  1 usage  Rui Teixeira
def __create_difficulty_tag(root: ET.Element[str], difficulty: Difficulty) -> None:
    question_element = ET.SubElement(root, question_tag, type=difficulty_tag)

    GeneratorExtensions.create_element_containing_text_element(
        question_element,
        difficulty_tag,
        text: f"{course_path_prefix}{difficulty.value}")
```

Figura 32: Função "create_difficulty_tag".

8. Foi definida função (método) estática *create_question_tag*.

Recebe como parâmetro a estrutura de elementos e a questão a incluir:

```
@staticmethod  1 usage  Rui Teixeira
def __create_question_tag(root: ET.Element[str], question: Question) -> None:
    question_element = ET.SubElement(root, question_tag, type=Type.translate_type(question.type).value)

    GeneratorExtensions.create_common_question_tags(question_element, question)
    GeneratorExtensions.create_type_specific_question_tags(question_element, question)
```

Figura 33: Função "create_question_tag".

9. Foi definida função *write_file* responsável pela escrita da “árvore de elementos” em ficheiro *XML*.

Recebe como parâmetro a estrutura de elementos.

Caso não existam, são criadas pastas necessárias:

```
def __write_file(self: Generator, root: ET.Element[str]) -> None:  1 usage  Rui Teixeira *
    if not self.quiz_path.upper().endswith(file_extension):
        raise ValueError("Invalid File: '{}'. Required XML.".format(self.quiz_path))

    if not os.path.exists(self.quiz_path):
        directory_path = os.path.dirname(os.path.abspath(self.quiz_path))
        os.makedirs(directory_path)

    with open(self.quiz_path, write_bytes_mode) as file:
        file.write(ET.tostring(root, encoding=default_encoding, xml_declaration=True))
```

Figura 34: Função "write_file".

1. Classe *Generator Extensions*

1. Foram definidos, em primeiro lugar, os campos referentes a elementos a ser preenchidos na estrutura a incluir em ficheiro *XML*, em formato aceite pelo Moodle:

```
text_tag = "text"
name_tag = "name"
question_text_tag = "questiontext"
```

Figura 35: Elementos constituintes de ficheiro resultante.

2. Foram definidos campos referentes ao elemento *answe*r (formato aceite pelo Moodle), incluindo cotação das respostas de tipo “Escolha Múltipla” e “Verdadeiro e Falso”:

```
answer_tag = "answer"
correct_answer_fraction = "100"
wrong_multiple_choice_answer_fraction = "0"
wrong_true_or_false_answer_fraction = "-25"
```

Figura 36: Campos referentes ao elemento”answer”.

3. Foram definidos campos referentes à aleatoriedade das respostas, em formato aceite pelo Moodle:

```
shuffle_answers_tag = "shuffleanswers"
shuffle_answers_active_tag = "1"
```

Figura 37: Campos referentes à aleatoriedade das respostas.

4. Foi definida a classe *GeneratorExtensions*:

```
class GeneratorExtensions:   12 usages   Rui Teixeira
```

Figura 38: Classe "Generator Extensions".

5. Foi definida a função (método) estática *create_element_containing_text_element*.

Recebe como parâmetro um elemento *question* (formato aceite pelo Moodle), qual o elemento a adicionar a esse e qual o texto a definir em elemento próprio.

É opcional o parâmetro *fraction*, útil na definição de pontuação:

```
@staticmethod  7 usages  Rui Teixeira
def create_element_containing_text_element(
    question_element: ET.Element[str],
    element_tag: str,
    text: str,
    fraction: str = None) -> None:
    if fraction is not None:
        element = ET.SubElement(question_element, element_tag, fraction=fraction)
    else:
        element = ET.SubElement(question_element, element_tag)

    ET.SubElement(element, text_tag).text = text
```

Figura 39: Função "create_element_containing_text_element".

6. Foi definida a função (método) estática *create_common_question_tags*.

Recebe como parâmetro um elemento *question* e um objeto *Question*.

É responsável por criar os elementos comuns a todos os elementos *question* (*name*, "*questiontext*" e "*shuffleanswers*"), ou seja, a todas as questões:

```
@staticmethod  1 usage  Rui Teixeira
def create_common_question_tags(question_element: ET.Element[str], question: Question) -> None:
    GeneratorExtensions.create_element_containing_text_element(question_element, name_tag, question.id)

    GeneratorExtensions.create_element_containing_text_element(
        question_element,
        question_text_tag,
        Format.format_text_to_display(question.text))

    ET.SubElement(question_element, shuffle_answers_tag).text = shuffle_answers_active_tag
```

Figura 40: Função "create_common_question_tags".

7. Foi definida a função (método) estática *create_type_specific_question_tags*.

Recebe como parâmetro um elemento *question* e um objeto *Question*.

É responsável pela construção dos elementos que correspondem às respostas corretas e erradas, ordenadamente:

```
@staticmethod  1 usage  Rui Teixeira
def create_type_specific_question_tags(question_element: ET.Element[str], question: Question) -> None:
    """
    Create type-specific question tags.
    Correct answer is specified first.

    :param ET.Element[str] question_element: The question element.
    :param Question question: The question object.
    """

    GeneratorExtensions.__create_correct_answer_tags(question_element, question)
    GeneratorExtensions.__create_wrong_answer_tags(question_element, question)
```

Figura 41: Função "create_type_specific_question_tags".

8. Foi definida a função (método) estática *create_wrong_answer_tags*.

Recebe como parâmetro um elemento *question* e um objeto *Question*.

É responsável pela construção dos elementos que correspondem às respostas erradas:

```
@staticmethod  1 usage  Rui Teixeira
def __create_wrong_answer_tags(question_element: ET.Element[str], question: Question) -> None:
    for wrong_answer in question.wrong_answers:
        match question.type.value:
            case MULTIPLE_CHOICE.value:
                GeneratorExtensions.create_element_containing_text_element(
                    question_element,
                    answer_tag,
                    wrong_answer.value,
                    wrong_multiple_choice_answer_fraction)

            case TRUE_OR_FALSE.value:
                GeneratorExtensions.create_element_containing_text_element(
                    question_element,
                    answer_tag,
                    wrong_answer.value.lower(),
                    wrong_true_or_false_answer_fraction)
```

Figura 42: Função "create_wrong_answer_tags".

9. Foi definida a função (método) estática *create_correct_answer_tags*.

Recebe como parâmetro um elemento *question* e um objeto *Question*.

É responsável pela construção dos elementos que correspondem às respostas corretas:

```
@staticmethod  1 usage  Rui Teixeira
def __create_correct_answer_tags(question_element: ET.Element[str], question: Question) -> None:
    for correct_answer in question.correct_answers:
        match question.type.value:
            case MULTIPLE_CHOICE.value:
                GeneratorExtensions.create_element_containing_text_element(
                    question_element,
                    answer_tag,
                    correct_answer.value,
                    correct_answer_fraction)

            case TRUE_OR_FALSE.value:
                GeneratorExtensions.create_element_containing_text_element(
                    question_element,
                    answer_tag,
                    correct_answer.value.lower(),
                    correct_answer_fraction)
```

Figura 43: Função "create_correct_answer_tags".

5. Classe *Format*

1. Foi definido, em primeiro lugar, o campo correspondente ao estilo de identação pretendida, aquando formatação de ficheiro *XML*:

```
indent_size = 4 * " "
```

Figura 44: Estilo de identação em formatação de ficheiro de tipo "XML".

2. Foram definidos campos correspondentes à representação de parágrafo em ficheiro *CSV* e respetivos símbolos em *Hypertext Markup Language* (*HTML*):

```
paragraph_placeholder = " ... "
open_paragraph_tag = "<p>"
close_paragraph_tag = "</p>"
```

Figura 45: Representação de parágrafo.

3. Foi definida a classe *Format*:

```
class Format:   7 usages   Rui Teixeira
```

Figura 46: Classe "Format".

4. Foi definida a função (método) *indent_element_tree*.

Recebe como parâmetro a estrutura de elementos.

É responsável pela identação:

```
@staticmethod   1 usage   Rui Teixeira
def indent_element_tree(root: ET.Element[str]) -> None:
    ET.indent(root, space=indent_size)
```

Figura 47: Função "indent_element_tree".

5. Foi definida a função (método) estática *format_text_to_display*.

Recebe como parâmetro o texto a ser formatado.

Tem como resultado o texto formatado, com devidos parágrafos, em formato *HTML*:

```
@staticmethod  1 usage  Rui Teixeira
def format_text_to_display(text: str) -> str:
    formatted_text_list = text.split(paragraph_placeholder)

    formatted_text = formatted_text_list[0]

    for paragraph in formatted_text_list[1:]:
        formatted_text += open_paragraph_tag + paragraph + close_paragraph_tag

    return formatted_text
```

Figura 48: Função "format_text_do_display".

6. Foi definida a função (método) estática *request_file_path*.

Recebe como parâmetro o nome do ficheiro a pedir, o tipo de ficheiro pretendido, a localização recomendada e se a mesma deve existir.

Tem como resultado, caso resposta válida, a localização do ficheiro.

É possível aceitar a resposta recomendada através da tecla *Enter*.

```
@staticmethod  2 usages  Rui Teixeira
def request_file_path(file_name: str, file_type: str, recommended_file_path: str, must_exist: bool = True) -> str:
    while True:
        file_path = input(f"Insert path to {file_name} (Recommended '{recommended_file_path}'): ").strip()

        if not file_path:
            file_path = recommended_file_path

        if not must_exist and file_path.upper().endswith(file_type.upper()):
            return file_path

        if must_exist and file_path.upper().endswith(file_type.upper()) and os.path.isfile(file_path):
            return file_path
```

Figura 49: Função "request_file_path".

5. Demonstração

```
Insert path to Questions Data (Recommended 'assets/QuestionsData.CSV'):
Insert path to Generated Quiz (Recommended 'quiz/Quiz.XML'):
Quiz Generated!
```

Figura 50: Demonstração de aplicação "Quiz Generator".

Nota: É possível a escolha da localização recomendada através da tecla *Enter*.

6. Moodle

1. É possível a submissão de todas as questões:

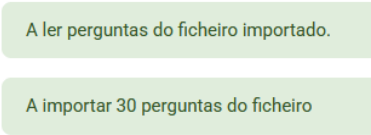

Figura 51: Submissão de todas as questões.

2. As categorias são criadas e as questões são organizadas por categoria:

Figura 52: Categorias disponíveis e questões agrupadas..

3. Exemplo de questão em teste:

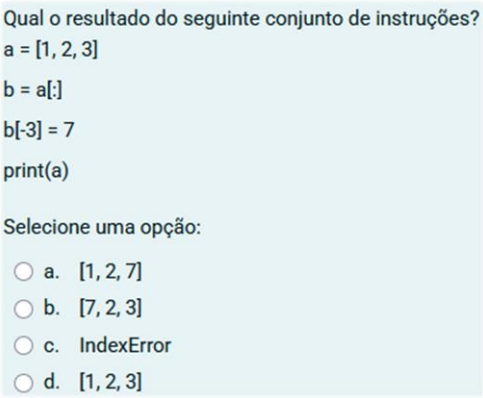

Figura 53: Questão em teste.